

Open in app ↗



Medium



Search



Write



S

Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)

VaultDoor:1 Walkthrough (pico CTF)



Hritom Bhattacharya

Follow

7 min read · Feb 9, 2024



36



Hii this is my second blog on reverse engineering based on vaultdoor series from pico CTF. We gonna be reverse today vaultdoor1 source code which is pretty easy. This is with 0.5 complexity level.

Let's Reverse.....

vault-door-1

It's really a easy code but with a basic and easy tricks.....

```
import java.util.*;

class VaultDoor1 {
    public static void main(String args[]) {
        VaultDoor1 vaultDoor = new VaultDoor1();
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.print("Enter vault password: ");
            String userInput = scanner.next();
        }
    }
}
```

```

String input = userInput.substring("picoCTF{".length(),userInput.length()-1);
if (vaultDoor.checkPassword(input,userInput)) {
    System.out.println("Access granted.");
} else {
    System.out.println("Access denied!");
}
}

// I came up with a more secure way to check the password without putting
// the password itself in the source code. I think this is going to be
// UNHACKABLE!! I hope Dr. Evil agrees...
//
// -Minion #8728
public boolean checkPassword(String password,String userInput) {

    return password.length() == 32 &&
        password.charAt(0) == 'd' &&
        password.charAt(29) == 'a' &&
        password.charAt(4) == 'r' &&
        password.charAt(2) == '5' &&
        password.charAt(23) == 'r' &&
        password.charAt(3) == 'c' &&
        password.charAt(17) == '4' &&
        password.charAt(1) == '3' &&
        password.charAt(7) == 'b' &&
        password.charAt(10) == '_' &&
        password.charAt(5) == '4' &&
        password.charAt(9) == '3' &&
        password.charAt(11) == 't' &&
        password.charAt(15) == 'c' &&
        password.charAt(8) == 'l' &&
        password.charAt(12) == 'H' &&
        password.charAt(20) == 'c' &&
        password.charAt(14) == '_' &&
        password.charAt(6) == 'm' &&
        password.charAt(24) == '5' &&
        password.charAt(18) == 'r' &&
        password.charAt(13) == '3' &&
        password.charAt(19) == '4' &&
        password.charAt(21) == 'T' &&
        password.charAt(16) == 'H' &&
        password.charAt(27) == '6' &&
        password.charAt(30) == 'f' &&
        password.charAt(25) == '_' &&
        password.charAt(22) == '3' &&
        password.charAt(28) == 'd' &&
        password.charAt(26) == 'f' &&
        password.charAt(31) == '4';

```

```
}  
}
```

Okayyy.... let's extract the interesting part (the main logic).... we will do this to really understand the workflow....

Part 1

```
String input = userInput.substring("picoCTF{".length(),userInput.length()-1);  
if (vaultDoor.checkPassword(input,userInput)) {  
    System.out.println("Access granted.");  
} else {  
    System.out.println("Access denied!");  
}
```

Okay let's go line by line.....

Line 1

This line is declering a string name input which extracting a specific string from the variable userInput:

1. Is a part of string value of userInput variable. This is achiving by `userInput.substring()`.
2. `"picoCTF{".length()` — -> This is denoting the length of `"picoCTF{"` which is 8. This also mean extraction will start from 9th position.
3. `userInput.length()-1` — -> This is declaring that the extraction will stop to the before the last character of string of the userInput variable or a digit earlier of the length of the userInput. For a example if length of the string

of userInput variable 12 it will stop at 11 (Let's say the string is going like ...Gh4R then it will remove R and will extract till 4).

Line 2,3,4&5

This line is calling checkPassword method against vaultdoor to check the value of input variable. (okay..... this is a security mechanism it's checking against vaultdoor object if it was not then it can be a broken authentication.). And giving output accordingly.

Part 2

```
public boolean checkPassword(String password,String userInput) {  
  
    return password.length() == 32 &&  
        password.charAt(0) == 'd' &&  
        password.charAt(29) == 'a' &&  
        password.charAt(4) == 'r' &&  
        password.charAt(2) == '5' &&  
        password.charAt(23) == 'r' &&  
        password.charAt(3) == 'c' &&  
        password.charAt(17) == '4' &&  
        password.charAt(1) == '3' &&  
        password.charAt(7) == 'b' &&  
        password.charAt(10) == '_' &&  
        password.charAt(5) == '4' &&  
        password.charAt(9) == '3' &&  
        password.charAt(11) == 't' &&  
        password.charAt(15) == 'c' &&  
        password.charAt(8) == 'l' &&  
        password.charAt(12) == 'H' &&  
        password.charAt(20) == 'c' &&  
        password.charAt(14) == '_' &&  
        password.charAt(6) == 'm' &&  
        password.charAt(24) == '5' &&  
        password.charAt(18) == 'r' &&  
        password.charAt(13) == '3' &&  
        password.charAt(19) == '4' &&
```

```
password.charAt(21) == 'T' &&  
password.charAt(16) == 'H' &&  
password.charAt(27) == '6' &&  
password.charAt(30) == 'f' &&  
password.charAt(25) == '_' &&  
password.charAt(22) == '3' &&  
password.charAt(28) == 'd' &&  
password.charAt(26) == 'f' &&  
password.charAt(31) == '4';  
  
}
```

Here, basically when passing the input to this method for checking the input and this method contains the password string into characters in jumbled up manner.

so for this we have to assemble the value of the character based on index value of charat()

so we get.....

```
picoCTF{d35cr4mbl3_th3_ch4r4cT3r5_f6daf4}
```

ust add a “}” in the end when you submit the flag the website check the full flag ...LOL!.... But when you gonna enter the flag in the program you can change the “}” with any one character because it is reducing one character to validate.... Let’s go forward with the program part because the main objective of this blog is to learning to reverse the program and vulnerabilities for secure coding practices and pentesting purposes.

Loopholes

I am not gonna continue this part for this series because all the loop holes in this series are same

so in short

Of course the basic vulnerability is encryption this will increase level wise let's not talk about this because this is a basic lab.

The main issue with this lab is a broken authentication. I mean to say that there password can be many more passwords, to bypass this CTF just because it is not authenticating "picoCTF{" and the last character.



It is just counting line length of "picoCTF{". So in this place if we change it to any random character let's say "hritom\$_" it will also validate this. Just like that the last character also get validated

Compile file reversing

Okay..... Let's suppose we got the compiled file in .class format for any reason and source code is missing Let's open it with Ghidra deassembler.

Go to the function in the symbol tree and goto the `main.java.lang.String[]_void` and in the decompile section you will notice

this.....

```
/* Flags:
    ACC_PUBLIC
    ACC_STATIC

    public static void main(java.lang.String[]) */

void main_java.lang.String[]_void(void)

{
    PrintStream pPVar1;
    String objectRef;
    int iVar2;
    int iVar3;
    String pSVar4;
    boolean bVar5;
    VaultDoor1 objectRef_00;
    Scanner objectRef_01;
    String objectRef_02;

    objectRef_00 = new VaultDoor1();
    objectRef_01 = new Scanner(System.in);
    pPVar1 = System.out;
    pPVar1.print("Enter vault password: ");
    objectRef = objectRef_01.next();
    pSVar4 = "picoCTF{";
    objectRef_02 = objectRef;
    iVar2 = pSVar4.length();
    iVar3 = objectRef.length();
    pSVar4 = objectRef_02.substring(iVar2, iVar3 + -1);
    bVar5 = objectRef_00.checkPassword(pSVar4, objectRef);
    if (bVar5 == false) {
        pPVar1 = System.out;
        pPVar1.println("Access denied!");
    }
    else {
        pPVar1 = System.out;
        pPVar1.println("Access granted.");
    }
    objectRef_01.close();
    return;
}
```

from here we will take our most likely interesting part to examine.....

```
PrintStream pPVar1;
String objectRef;
int iVar2;
int iVar3;
String pSVar4;
boolean bVar5;
VaultDoor1 objectRef_00;
Scanner objectRef_01;
String objectRef_02;

objectRef_00 = new VaultDoor1();
objectRef_01 = new Scanner(System.in);
pPVar1 = System.out;
pPVar1.print("Enter vault password: ");
objectRef = objectRef_01.next();
pSVar4 = "picoCTF{";
objectRef_02 = objectRef;
iVar2 = pSVar4.length();
iVar3 = objectRef.length();
pSVar4 = objectRef_02.substring(iVar2, iVar3 + -1);
bVar5 = objectRef_00.checkPassword(pSVar4, objectRef);
if (bVar5 == false) {
    pPVar1 = System.out;
    pPVar1.println("Access denied!");
}
else {
    pPVar1 = System.out;
    pPVar1.println("Access granted.");
}
```

Okay let's try to understand this....

1. here pSVar4 is a string that is "picoCTF{" and iVar2 is a integer which is the length of pSVar2 [iVar2 = pSVar4.length();].
2. iVar3 is also a integer which is taking the integer value of the length of objectRef. objectRef is a string is the string which is taking the input as

you can see.....

```
objectRef_01 = new Scanner(System.in);  
  
objectRef = objectRef_01.next();
```

3. in the next line re-writing the value of pSVar4 which is extracting a specific part of string from objectRef_02.

```
pSVar4 = objectRef_02.substring(iVar2,iVar3 + -1);
```

here objectRef_02 is the objectRef itself. And [iVar2,iVar3 + -1] this part defines length of “picoCTF{” which is 8 to the 2nd last character (start count from 9th character).

4. next line is just calling chechPassword against objectRef_00 with pSVar4 argument.

5. And this part is just conditional statement

```
if (bVar5 == false) {  
    pPVar1 = System.out;  
    pPVar1.println("Access denied!");  
}  
else {  
    pPVar1 = System.out;
```

```
pPVar1.println("Access granted.");
}
```

Now in the symbol tree in the function go to
checkPassword_java.lang.String_boolean function

```
/* Flags:
   ACC_PUBLIC

   public boolean checkPassword(java.lang.String, java.lang.String) */
undefined4 checkPassword_java.lang.String_java.lang.String_boolean(undefined4 pa
{
    int iVar1;
    char cVar2;

    iVar1 = param1.length();
    if ((((((((((iVar1 == 0x20) && (cVar2 = param1.charAt(0), cVar2 == 'd')) &&
        (cVar2 = param1.charAt(0x1d), cVar2 == 'a')) &&
        (((cVar2 = param1.charAt(4), cVar2 == 'r' && (cVar2 = param1.charAt(2)
        ((cVar2 = param1.charAt(0x17), cVar2 == 'r' &&
        ((cVar2 = param1.charAt(3), cVar2 == 'c' && (cVar2 = param1.charAt(0
        )))))) && (cVar2 = param1.charAt(1), cVar2 == '3')) &&
        (((((cVar2 = param1.charAt(7), cVar2 == 'b' && (cVar2 = param1.charAt(10
        (cVar2 = param1.charAt(5), cVar2 == '4')) &&
        ((cVar2 = param1.charAt(9), cVar2 == '3' && (cVar2 = param1.charAt(0xb
        && (cVar2 = param1.charAt(0xf), cVar2 == 'c')))) &&
        (((cVar2 = param1.charAt(8), cVar2 == 'l' && (cVar2 = param1.charAt(0xc),
        (((cVar2 = param1.charAt(0x14), cVar2 == 'c' &&
        (((cVar2 = param1.charAt(0xe), cVar2 == '_' && (cVar2 = param1.charAt(
        && (cVar2 = param1.charAt(0x18), cVar2 == '5')))) &&
        (((cVar2 = param1.charAt(0x12), cVar2 == 'r' && (cVar2 = param1.charAt(
        && (cVar2 = param1.charAt(0x13), cVar2 == '4')))))))) &&
        (((cVar2 = param1.charAt(0x15), cVar2 == 'T' && (cVar2 = param1.charAt(0x1
        && ((cVar2 = param1.charAt(0x1b), cVar2 == '6' &&
        (((cVar2 = param1.charAt(0x1e), cVar2 == 'f' &&
        (cVar2 = param1.charAt(0x19), cVar2 == '_')) &&
        (cVar2 = param1.charAt(0x16), cVar2 == '3')))))))) &&
        (((cVar2 = param1.charAt(0x1c), cVar2 == 'd' && (cVar2 = param1.charAt(0x1a
        (cVar2 = param1.charAt(0x1f), cVar2 == '4')))) {
        return 1;
    }
```

```
}  
return 0;  
}
```

1. The index of charat() using hex value.

```
iVar1 = param1.length();  
if (((((((iVar1 == 0x20)
```

This line is just measuring the length of the input and in the next line this particular portion is checking that value of iVar1 is 32 or not (0x20 is the hex representation of 32).

If you hover mouse pointer or right click on the hex values you will notice that it will show you the decimal value.

In this conditional statement cVar2 is checking the character of the input string in a jumbled up manner.

like this.....

```
cVar2 = param1.charAt(0x1d), cVar2 == 'a'
```

Basically, cVar2 is taking the value of the character of the string which is equal to the hex representation of the index of charAt() and (cVar2 == 'a') in

this portion just checking the value of cVar2 with any constant character.

There is more 7 to go with will go forward with them in future.

I just shared my perspective to understand a program Thank You.

Happy Hacking!

[Reverse Engineering](#)[Source Code](#)[Java](#)[Ctf](#)[Writeup](#)

Written by Hritom Bhattacharya

15 followers · 10 following

[Follow](#)

No responses yet



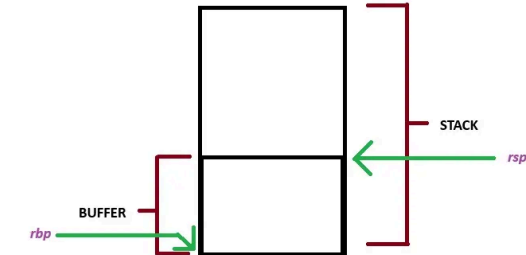
Suzieq

What are your thoughts?

More from Hritom Bhattacharya

```
1 public static void main(String args[]) {
2     VaultDoorTraining vaultDoor = new VaultDoorTraining();
3     try (Scanner scanner = new Scanner(System.in)) {
4         System.out.print("Enter vault password: ");
5         String userInput = scanner.next();
6         String input = userInput.substring("picoCTF".length(), userInput.length()-1);
7         if (vaultDoor.checkPassword(input)) {
8             System.out.println("Access granted.");
9         } else {
10            System.out.println("Access denied.");
11        }
12    }
13 }
14
15 // The password is below. Is it safe to put the password in the source code?
16 // What if somebody stole our source code? Then they would know what our
17 // password is. Hmm... I will think of some ways to improve the security.
```

hritom@vms14w3 in /mnt/Documents/program/vaultdoor via zsh 0.2 took 22s



 Hritom Bhattacharya

Basic Reverse Engineering: Vault Door: training Walkthrough (pico...

Hii I am Hritom Bhattacharya a security enthusiast. And this is my first blog , I am...

Jan 26, 2024  3



 Hritom Bhattacharya

Why scanf() function is just like your Ex?!

Hey, ever your ex betrayed you? :)

Nov 17, 2025



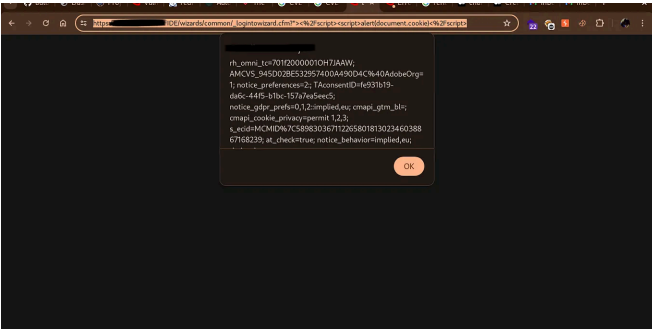


 Hritom Bhattacharya

Dirtypipe—which gives you a special “geass” power

Heyy! everyone ... I was playing some CTF where I got this this CVE which cause...

Sep 12, 2025  1



 Hritom Bhattacharya

Story of Hall of Fame in Red Hat Linux

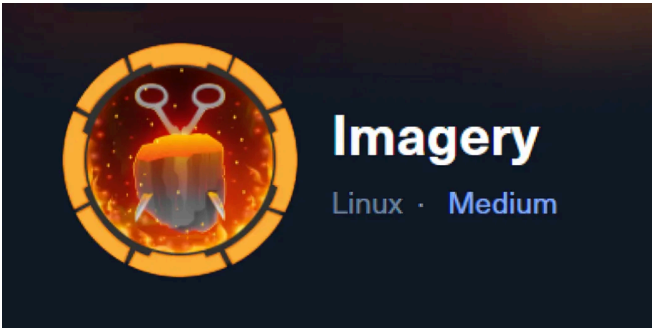
Hello! I’m a independent bug bounty hunter rather just a starting with bug bounty a...

May 13, 2024  7



See all from Hritom Bhattacharya

Recommended from Medium



ad-dump 

✗OR: PRINCE NIYONSHUTI N.

scription

come to the challenge! In this challenge, you will explore a web application find an endpoint that exposes a file containing a hidden flag.

application is a simple blog website where you can read articles about ous topics, including an article about API Documentation. Your goal is to lore the application and find the endpoint that generates files holding the er's memory, where a secret flag is hidden.

itional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: NOT_RUNNING

[Launch Instance](#)

Hints 

[1](#) [2](#)



Oussama Djelloul

Hack The Box — Imagery: A Complete Walkthrough

Executive Summary {#executive-summary}

Nov 15, 2025

👏 18

💬 1



Saumya Srivastava

Flag in Flame 🔥 — How a Giant Log File Hid a Secret Image and a Flag

by Saumya Srivastava | Forensics | picoCTF 2025

Nov 5, 2025

💬 2



SSTI1

Easy Web Exploitation picoCTF 2025 browser_webshell_solvable

AUTHOR: VENAX

Description

I made a cool website where you can announce whatever you want! Try it out!

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.
Its current status is: **NOT_RUNNING**

[Launch Instance](#)

Hints



Maram Raboudi

PicoCTF 2025 | All Web Exploitation Challenges Solution

Challenge 1 | SSTI1 (easy)



Walter Moar

Writeup for picoCTF challenge “head-dump”

Explore API documentation to find a heap snapshot endpoint and learn why server...



Mar 12

👏 12

💬 1



AUTHOR: YAHAYA MEDDY

Description

A university's online registration portal asks students to upload their ID cards for verification. The developer put some filters in place to ensure only image files are uploaded but are they enough? Take a look at how the upload is implemented. Maybe there's a way to slip past the checks and interact with the server in ways you shouldn't.

You can access the web application at [here!](#)

This challenge launches an instance on demand.
Its current status is: **RUNNING**
Instance Time Remaining: **14:44**

[Restart Instance](#)

Hints



4mN3s14

picoMINI CTF

byp4ss3d: How I Tricked a University Portal into Running My Homework (and Stole the...

Oct 16, 2025

👏 11



crypto / Simple Encryption

Flag Hacks 328 Hacks

First Blood 1337devil

Simple Encryption **Easy**

est a simple encryption algorithm based on basic arithmetic operations.



3mksaad1600

CTF Writeup: Simple Encryption

Flagyard cryptography challenge.

Sep 24, 2025



2



1



Nov 14, 2025



1



See more recommendations